1. What is CRUD in FastAPI?
CRUD operations are the basic functions used to manage data in an application.
In FastAPI, CRUD operations are implemented using API endpoints and HTTP methods.
CRUD stands for:

- C → Create(POST Method)
  Create operation is used to add new data into the system or database.
- R → Read(GET Method)
  Read operation is used to retrieve or fetch data from the system.
- U → Update(PUT Method)
  Update operation is used to modify existing data in the system.
- D → Delete(DELETE Method)
  Delete operation is used to remove data from the system.

2. Why do we use the POST method for inserting data?
The POST method is used to send new data from the client to the server.
In FastAPI, POST is mainly used for the Create operation in CRUD.
Purpose of POST Method
The POST method is used to:

- Add new records
- Send data to the server
- Store information in a database or list

3. Difference between PUT and PATCH
Both PUT and PATCH methods are used to update data in APIs.
In FastAPI, they are commonly used in CRUD operations.
However, there is an important difference between them.

PUT	PATCH
Full update	Partial Update
Replaces complete object	Updates selected fields
Requires all fields	Requires only changed fields
Used for complete modification	Used for small changes

4. What is the path parameter?
A Path Parameter is a value passed inside the URL path to identify a specific resource.
In FastAPI, path parameters are used to access specific data such as a student ID, product ID, or user ID.

Path parameters help to:

- Access specific records
- Update specific data
- Delete specific data
- Identify resources uniquely

5. What is the request body in FastAPI?

A Request Body is the data sent by the client to the server inside an HTTP request in JSON format for creating or updating resources. FastAPI automatically validates this data using Pydantic models.

In FastAPI, request bodies are mainly used with:

- POST
- PUT
- PATCH

methods to send data such as user details, student information, login data, etc.

Request body is used to:

- Send large data
- Insert records
- Update records
- Transfer structured information

6. Why do we use Pydantic models?

In FastAPI, Pydantic models are used for:

- data validation,
- data parsing,
- and defining the structure of request and response data.

Pydantic helps FastAPI ensure that the user sends correct and valid data.

7. What status code is returned for successful deletion?

In FastAPI, the status code commonly returned for successful deletion is:

`200 OK` means:

- Request was successful
- Student record was deleted successfully

`204 No Content` is used when:

- deletion is successful,
- but no response body is returned.

8. How can we check whether a student exists before updating?

Before updating a student record in FastAPI, we must first check whether the student ID already exists in the student list.

This prevents updating a non-existing student.

Without checking:

- the API may try to update invalid data,
- or cause errors.

Checking ensures:

- only existing students are updated.

9. What happens if duplicate IDs are inserted?

In FastAPI, if duplicate IDs are inserted without validation, multiple students can have the same ID.

This creates data inconsistency and confusion.

If duplicate IDs are inserted:

- data confusion occurs,
- updates and deletions may become incorrect.

Therefore, duplicate ID checking should be performed before inserting new student records.

10. How does FastAPI automatically generate Swagger UI?

FastAPI automatically reads your code and creates a documentation webpage (`/docs`) using Swagger UI by using

- Python type hints,
- Pydantic models,
- and OpenAPI schema.

So, developers can test and understand APIs directly from the browser.

When we create API endpoints in FastAPI, it automatically creates API documentation without writing extra code.

Swagger UI is an interactive API documentation page.

It helps developers:

- view APIs,
- test APIs,
- send requests,
- and see responses directly from the browser.